

บทที่ 3 · CONTAINER ORCHESTRATION

Introduction to Kubernetes

ทำความรู้จักสถาปัตยกรรมและองค์ประกอบหลักของ Kubernetes

1. Kubernetes คืออะไร

Kubernetes (มักเรียกย่อว่า K8s) เป็น open-source platform สำหรับจัดการ (orchestrate) container ในระดับ production พัฒนาโดย Google และปัจจุบันดูแลโดย Cloud Native Computing Foundation (CNCF) Kubernetes ช่วยให้ทีมสามารถ deploy, scale และดูแล application ที่รันเป็น container ได้แบบอัตโนมัติ โดยไม่ต้องจัดการเครื่องแต่ละเครื่องด้วยมือ

แทนที่จะบอกว่า "รัน container ตัวนี้บนเครื่องนั้น" นักพัฒนาเพียงประกาศ *สถานะที่ต้องการ* (desired state) เช่น "ต้องการให้ service นี้มี 3 replicas ทำงานตลอดเวลา" แล้ว Kubernetes จะดูแลให้ระบบเป็นไปตามที่ประกาศไว้เอง หาก container ตัวใดตัวหนึ่งตาย ระบบจะสร้างใหม่ให้อัตโนมัติ

ทำไมเราถึงต้องการ Kubernetes

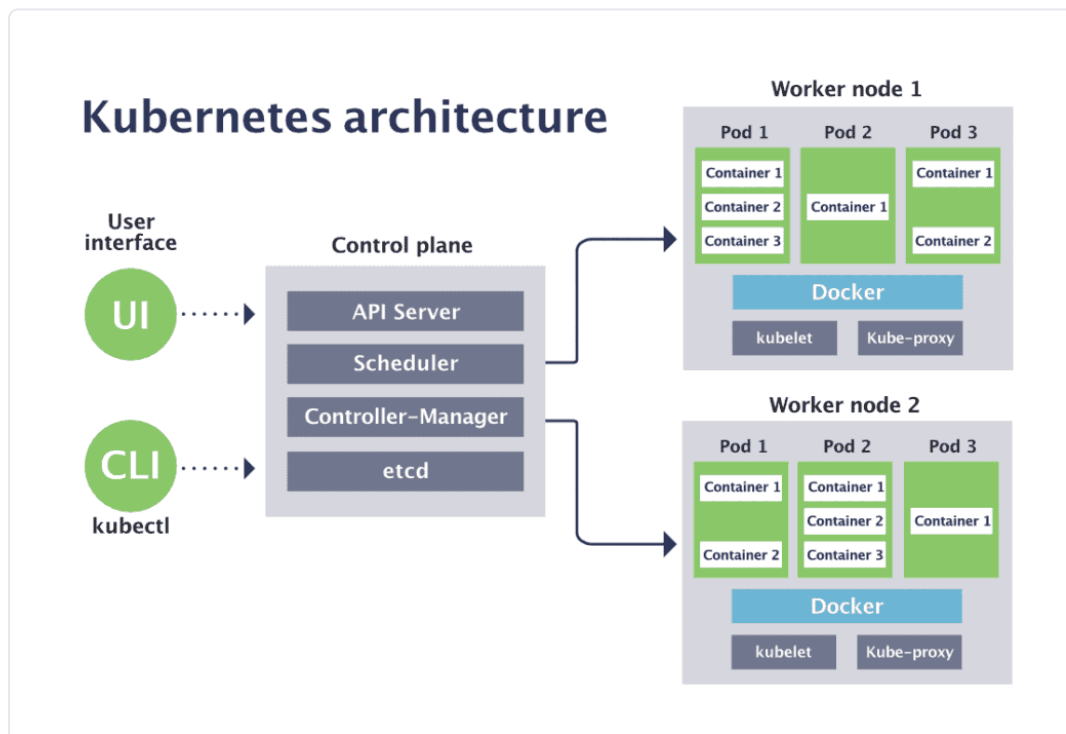
- **Self-healing:** ตรวจสอบสุขภาพ container และ restart เมื่อมีปัญหา
- **Auto-scaling:** เพิ่ม/ลดจำนวน replicas ตามภาระงาน
- **Service discovery & load balancing:** กระจาย traffic ระหว่าง pod โดยอัตโนมัติ
- **Rolling updates:** อัปเดต application แบบ zero-downtime
- **Declarative configuration:** อธิบายผลลัพธ์ที่ต้องการเป็นไฟล์ YAML แทนการเขียน script

เกร็ดน่ารู้

ชื่อ "Kubernetes" มาจากภาษากรีก แปลว่า "นายท้ายเรือ" (helmsman) — สื่อถึงบทบาทของมันในการควบคุมเรือ (container) ให้แล่นไปในทิศทางที่ต้องการ ส่วนตัวเลข 8 ในคำว่า K8s มาจากจำนวนตัวอักษรระหว่าง K และ s

2. ภาพรวมสถาปัตยกรรม

Kubernetes cluster หนึ่ง ๆ ประกอบด้วยเครื่องหลายเครื่องที่ทำหน้าที่แตกต่างกัน แบ่งเป็นสองบทบาทหลัก คือ **Control Plane** (สมองของระบบ) และ **Worker Nodes** (แรงงานที่รัน workload จริง) ผู้ใช้งานสื่อสารกับ cluster ผ่าน UI หรือ CLI (`kubectl`) โดยส่งคำสั่งไปยัง API Server ซึ่งเป็นประตูทางเข้าเดียวของทั้งระบบ



รูปที่ 3.1 — องค์ประกอบหลักของ Kubernetes cluster: ฝั่ง Control Plane และ Worker Nodes

3. องค์ประกอบของ Control Plane

Control Plane ทำหน้าที่ตัดสินใจว่า workload ตัวไหนควรไปรันที่ node ไหน รวมถึงคอยเฝ้าดูสถานะของ cluster และปรับให้ตรงกับสิ่งที่ผู้ใช้ประกาศไว้ตลอดเวลา

Component	หน้าที่
API Server	ประตูทางเข้าของทั้ง cluster รับคำสั่งจาก kubectl หรือ UI แล้วส่งต่อไปยังส่วนอื่น เป็นจุดเดียวที่ทุกคนคุยด้วย
Scheduler	ตัดสินใจว่า pod ที่เพิ่งถูกสร้างขึ้นควรไปรันที่ worker node ไหน โดยพิจารณาจากทรัพยากรที่ว่างและ constraint ต่าง ๆ
Controller Manager	รัน controller loop ที่คอยเปรียบเทียบ desired state กับ current state และทำการแก้ไขให้ตรงกัน เช่น สร้าง pod ใหม่แทนตัวที่ตาย
etcd	Key-value store ที่เก็บสถานะทั้งหมดของ cluster เปรียบเสมือน "ความทรงจำ" ของ Kubernetes หาก etcd เสียหาย cluster จะไม่สามารถทำงานได้

4. องค์ประกอบของ Worker Node

Worker node คือเครื่องที่รัน application จริง ๆ ในรูปแบบของ container โดยแต่ละ node จะมี component พื้นฐานดังนี้

Component	หน้าที่
kubelet	Agent ที่รันอยู่บนทุก node คอยรับคำสั่งจาก Control Plane และดูแลให้ container ที่ควรรันบน node นี้ทำงานอยู่จริง
kube-proxy	จัดการ network rule บน node เพื่อให้ traffic เข้าถึง pod ได้ ไม่ว่า pod นั้นจะอยู่ที่ node ใดใน cluster
Container Runtime	Software ที่รัน container จริง เช่น Docker, containerd หรือ CRI-O ทำหน้าที่ pull image และ start/stop container ตามคำสั่งของ kubelet

Pod: หน่วยเล็กที่สุดของ Kubernetes สำคัญ

Pod คือหน่วยย่อยที่สุดที่ Kubernetes จัดการโดยตรง หนึ่ง pod สามารถมีได้ตั้งแต่หนึ่ง container ขึ้นไป โดย container ภายใน pod เดียวกันจะแชร์ network namespace และ storage volume ร่วมกัน สิ่งนี้ทำให้ container ที่เกี่ยวข้องกันอย่างใกล้ชิด (เช่น application + sidecar สำหรับ logging) สามารถทำงานเสมือนอยู่บนเครื่องเดียวกันได้

จำไว้ให้แม่น

Kubernetes ไม่ deploy container โดยตรง — แต่ deploy pod ที่ห่อ container อีกที หาก pod ถูกลบและสร้างใหม่ จะได้ IP ใหม่เสมอ ดังนั้นการเข้าถึง pod จากภายนอกจึงต้องใช้ Service เป็นตัวกลาง (จะเรียนในบทถัดไป)

5. สรุปประเด็นสำคัญ

- Kubernetes เป็นระบบ orchestrate container แบบ declarative — เรียบอกว่า "อยากได้อะไร" ไม่ใช่ "ทำยังไง"
- Cluster แบ่งเป็น Control Plane (สมอง) และ Worker Node (แรงงาน)
- API Server เป็นประตูเดียวสำหรับสื่อสารกับ cluster
- etcd เก็บสถานะทั้งหมด — เป็นหัวใจของ cluster
- Pod คือหน่วยที่เล็กที่สุดที่ K8s จัดการ ไม่ใช่ container

Lab ต่อไป — Lab 3.1: Deploy pod แรกของคุณ

ในแล็บถัดไป คุณจะได้อลอง **kubectl apply** ไฟล์ YAML เพื่อสร้าง pod แรกบน sandbox cluster พร้อมทั้งใช้ **kubectl get pods** และ **kubectl describe** เพื่อสำรวจสิ่งที่ Kubernetes ทำภายใต้ฉาก